

On the Evaluation of Large Language Models in Unit Test Generation

Lin Yang*
College of Intelligence and
Computing, Tianjin
University
China
linyang@tju.edu.cn

Chen Yang*
College of Intelligence and
Computing, Tianjin
University
China
yangchenyc@tju.edu.cn

Shutao Gao
School of Future
Technology, Tianjin
University
China
gaoshutao@tju.edu.cn

Weijing Wang
College of Intelligence and
Computing, Tianjin
University
China
wangweijing@tju.edu.cn

Bo Wang
School of Computer and
Information Technology,
Beijing Jiaotong University
China
wangbo_cs@bjtu.edu.cn

Qihao Zhu
Key Laboratory of HCST,
MoE DCST, Peking
University
China
zhuqh@pku.edu.cn

Xiao Chu
Huawei Cloud Computing
Co. Ltd.
China
chuxiao1@huawei.com

Jianyi Zhou
Huawei Cloud Computing
Co. Ltd.
China
zhoujianyi2@huawei.com

Guangtai Liang
Huawei Cloud Computing
Co. Ltd.
China
liangguangtai@huawei.com

Qianxiang Wang
Huawei Cloud Computing
Co. Ltd.
China
wangqianxiang@huawei.com

Junjie Chen[†]
College of Intelligence and
Computing, Tianjin
University
China
junjiechen@tju.edu.cn

ABSTRACT

Unit testing is an essential activity in software development for verifying the correctness of software components. However, manually writing unit tests is challenging and time-consuming. The emergence of Large Language Models (LLMs) offers a new direction for automating unit test generation. Existing research primarily focuses on closed-source LLMs (e.g., ChatGPT and CodeX) with fixed prompting strategies, leaving the capabilities of advanced open-source LLMs with various prompting settings unexplored. Particularly, open-source LLMs offer advantages in data privacy protection and have demonstrated superior performance in some tasks. Moreover, effective prompting is crucial for maximizing LLMs' capabilities. In this paper, we conduct the first empirical study to fill this gap, based on 17 Java projects, five widely-used open-source LLMs with different structures and parameter sizes, and comprehensive evaluation metrics. Our findings highlight the significant influence of various prompt factors, show the performance of open-source LLMs compared to the commercial GPT-4 and the traditional Evosuite, and identify limitations in LLM-based unit test generation.

*Both authors contributed equally to this research.

[†]Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE '24, October 27-November 1, 2024, Sacramento, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1248-7/24/10

<https://doi.org/10.1145/3691620.3695529>

We then derive a series of implications from our study to guide future research and practical use of LLM-based unit test generation.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**.

KEYWORDS

Large Language Model, Unit Test Generation, Empirical Study

ACM Reference Format:

Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. 2024. On the Evaluation of Large Language Models in Unit Test Generation. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, October 27-November 1, 2024, Sacramento, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3691620.3695529>

1 INTRODUCTION

Unit testing focuses on verifying the functionality of each individual program component (such as each method) in order to ensure it works as intended. Hence, writing high-quality unit tests is crucial, which facilitates developers to catch defects early and diagnose them efficiently. However, manually writing these tests is time-consuming and tedious, which could significantly escalate the cost of software development [36]. To reduce this effort, several approaches have been proposed to automatically generate unit tests. Traditional approaches adopted symbolic execution [62, 87], evolutionary algorithms [20], or model checking [18, 21] to automatically generate unit tests. While they can generate tests with promising coverage, they still fall short of the high utility achieved

by human-written tests. For example, the tests can be difficult to maintain and understand due to the code smell issue, such as lack of meaningful identifiers [59]. To alleviate this problem, several deep-learning-based (DL-based) approaches have been proposed [8, 74]. They collected a large corpus of unit tests with their corresponding focal method to build DL models, thereby enhancing their test generation effectiveness. However, due to limitations in the scale of models and training data, they often struggle to fully understand the code's intent and generate high-quality tests.

Although existing approaches have made significant progress, automatically generating unit tests still faces the following challenges: (1) *Generating syntactically correct tests*. Unit test generation can be treated as a sub-problem of code generation, and thus also suffers from the general problem of ensuring that the generated code conforms to language's grammar rules. (2) *Generating effective tests*. High-quality unit tests should thoroughly explore the behaviour of the target program component, which requires the approach to understand code intent and structures and thus generate tests with high test coverage and defect detection ability. (3) *Generating maintainable tests*. In software evolution, we prefer unit tests that are easy to read and change, which requires the approach to generate tests following the code practice and smell well.

Recently, Large Language Models (LLMs), with supreme abilities in code understanding and natural language processing, have shown potential in code generation tasks. Researchers also proposed to leverage the most popular LLMs, i.e., GPT-3.5 or GPT-4, to automatically generate unit tests [68, 88, 92]. However, these LLMs are commercial and closed-source, which presents practical challenges due to concerns over data security and the costs associated with API usage. Additionally, these existing studies predominantly relied on fixed prompting strategies based on prior experience, neglecting the potential influence of various prompting factors, such as prompt design choices and in-context learning (ICL) methods. As shown in existing studies, prompting is crucial in maximizing the capabilities of LLMs [42, 49, 81, 93], and thus this lack of in-depth analysis on these factors could significantly limit the development of actionable guidelines for optimizing unit test generation with LLMs [64, 67]. Moreover, with the rapid emergence of open-source code LLMs and various prompting methods, their effectiveness for unit test generation remains largely unexplored. Therefore, it is urgent to investigate the effectiveness of advanced open-source LLMs in unit test generation with various prompting methods.

In this work, we conducted the first extensive study to achieve the above-mentioned goal based on 17 Java projects from the Defects4J 2.0 benchmark [32]. Particularly, we evaluated five open-source code LLMs, which are built on top of the widely-used CodeLlama [63] and DeepSeek-Coder [15] structures with diverse model scales ranging from 7B to 34B. Based on the extensive experiments under the guidance of our elaborately-designed four research questions (RQs), we obtain a series findings about LLM-based unit test generation and deliver the corresponding actionable guidelines for future research and practice use in this field, which are mainly summarized as follows:

- (1) The prompt design (the description style and selected code features) is crucial to the effectiveness of LLMs in unit test generation. It is recommended to align the description style with

the training data and choose code features considering the LLMs' code comprehension ability and the space left for generating unit tests. Notably, including other methods defined in the target class (except the focal method) negatively impacts the overall effectiveness of LLM-based unit test generation due to their extensive length.

- (2) The conclusions drawn from open-source LLMs in other tasks do not necessarily generalize to unit test generation, including dominance relationships among studied LLMs. However, all studied LLMs, including the state-of-the-art GPT-4, underperform traditional Evosuite in terms of test coverage. This is primarily due to the large percentage of syntactically invalid unit tests generated by LLMs, a result of LLMs' hallucination. Therefore, effective solutions are needed, e.g., designing post-processing rules to fix common syntactic issues.
- (3) Despite their effectiveness in other tasks, directly adapting the Chain-of-Thoughts (CoT) and Retrieval Augmented Generation (RAG) methods for unit test generation does not improve effectiveness and may even reduce it in some cases. CoT is primarily limited by the LLMs' code comprehension ability, while RAG is constrained by the significant gap between the retrieved unit tests and those that LLMs excel at generating. Special design for the use of ICL methods in unit test generation is required.
- (4) The defect detection ability of LLM-generated unit tests is limited, primarily due to their low validity. Although valid unit tests are generated by LLMs for many defects, a significant number of defects remain undetected, mainly because the tests fail to produce the specific inputs necessary to trigger these defects. Therefore, designing effective mutation strategies for the inputs within generated unit tests could further improve defect detection effectiveness.

In summary, this paper makes the following contributions:

- We performed the first empirical study to evaluate open-source LLMs in unit test generation. We studied five powerful code LLMs, ranging from 7B to 34B parameters, on 17 Java projects from the well-known Defects4J benchmark. Our experiments required around 3,000 NVIDIA A100 GPU-hours, underscoring the scale and intensity of our study in advancing LLM-based unit test generation.
- We evaluated LLM-based unit test generation from four aspects defined by our research questions, including the influence of prompt design, the influence of ICL methods, the comparison among open-source LLMs, the commercial GPT-4, and traditional Evosuite, and the evaluation in terms of diverse metrics (including syntactic validity, test coverage, and defect detection).
- We summarized nine major findings from our extensive study and delivers a series of implications for guiding future research and practice use of LLM-based unit test generation.

2 STUDY DESIGN

In general, when using LLMs to address some code-related task, it involves three main aspects: an LLM, a prompt as the input of the LLM, and an in-context (ICL) method employed to improve LLMs in addressing the task, and thus it is important to investigate these

aspects in LLM-based unit test generation. Furthermore, as the goal of unit testing is to detect defects at an early stage, the defect detection ability of the generated unit tests by LLMs is also crucial. In this work, to conduct a comprehensive study, the workflow of our study consists of four parts that target each aspect of LLM-based unit test generation as discussed above.

2.1 Research Questions

We investigated the unit test generation effectiveness of LLMs by answering the following research questions (RQs):

- **RQ1:** How does prompt design affect the effectiveness of LLMs in unit test generation?
- **RQ2:** How do open-source LLMs perform in unit test generation compared to GPT-4 and Evosuite?
- **RQ3:** How do in-context learning methods affect the effectiveness of LLM-based unit test generation?
- **RQ4:** How effective are the unit tests generated by LLMs in terms of defect detection?

In RQ1, we investigated the effectiveness of each studied open-source LLM in unit test generation by designing different prompts. This helps investigate the influence of different prompt designs on this task and also helps determine how to design a prompt to make LLMs achieve the best effectiveness. In RQ2, we compared the studied open-source LLMs in unit test generation based on the best prompt design we explored in RQ1, and employed the state-of-the-art commercial LLM (i.e., GPT-4) and the widely-studied traditional approach (i.e., Evosuite) as reference. This helps understand the current situation of these open-source LLMs in unit test generation. In RQ3, we incorporated two widely-used ICL methods to improve the usage of these studied LLMs respectively, and then revisited the effectiveness of these LLMs in unit test generation. This is helpful to explore whether the state-of-the-art ICL methods in general tasks can help improve the effectiveness of LLMs in the specific task of unit test generation.

The former three RQs correspond to the above-mentioned three aspects affecting the effectiveness of LLMs in addressing some tasks, which adopted the widely-used evaluation metrics in existing studies on LLM-based unit test generation (i.e., syntactic validity and test coverage). RQ4 further complements them by considering another key ability (i.e., the defect detection ability) of unit tests. That is, in RQ4, we investigated the effectiveness of these studied LLMs in terms of fault detection with the corresponding automatically generated unit tests.

2.2 Studied LLMs

First, we selected a set of representative code LLMs for investigation in our study. Our selection process started with the leaderboard of code LLMs hosted on Hugging Face [4]. We first sorted code LLMs based on their scores and found that the most powerful LLMs shared two structures: CodeLlama and DeepSeekCoder. Then, we determined a set of code LLMs with different scales for each structure. They are CodeLlama-7B-Instruct, CodeLlama-13B-Instruct, Phind-CodeLlama-34B-v2, DeepSeekCoder-6.7B-Instruct, and DeepSeekCoder-33B-Instruct. Besides, we used the state-of-the-art commercial LLM (i.e., GPT-4 [6]), as a reference due to its well-known effectiveness.

CodeLlama refers to a series of code-specific LLMs evolved from Llama 2 [73], available in three sizes (i.e., 7B, 13B, and 34B) and three variants (i.e., base model, Python fine-tuned and instruction-tuned models). **Phind** models are the fine-tuned versions of CodeLlama-34B, leveraging an internal Phind corpus and instruction-tuning, and they have outperformed other models in CodeLlama series. In our study, we selected three instruction-tuned LLMs: CodeLlama-7B-Instruct, CodeLlama-13B-Instruct, and Phind-CodeLlama-34B-v2, as instruction-tuned models tend to exhibit superior performance as shown in the existing work [91]. For ease of presentation, we call them **CL-7B**, **CL-13B**, and **PD-34B**, respectively.

DeepSeekCoder comprises a series of code LLMs trained on a repository-level code corpus, including both pre-trained and instruction-tuned models. These models come in various sizes, including 1.3B, 5.7B, 6.7B, and 33B. To ensure fair comparisons with our selected CodeLlama models, we selected DeepSeekCoder-6.7B-Instruct and DeepSeekCoder-33B-Instruct since they have comparable parameter sizes with CL-7B and PD-34B. We call them **DC-7B** and **DC-33B**, respectively.

ChatGPT includes influential commercial closed-source LLMs developed by OpenAI, such as GPT-3.5 and GPT-4. These models are pre-trained on trillions of tokens and further enhanced through instruction-tuning and reinforcement learning with human feedback [94]. They have shown exceptional performance in various tasks [88, 92]. Among the ChatGPT series, GPT-4 is the most advanced and outperforms its predecessors. Therefore, we selected **GPT-4** as the representative closed-source LLM for reference.

2.3 Prompt Design

Prompts are crucial for an LLM as they contain the context, instruct the task, and largely influence the model's performance [9, 23, 39, 64]. In general, when handling code-related tasks, the prompt should include proper code features, and the task and these code features should be described in a proper style, which can facilitate the LLM understanding and thus address this task. That is, when designing a prompt, we should take two factors into consideration: (1) which style is used to describe the task and these code features, and (2) which code features should be included.

Description Style. There are two widely-used description styles in addressing code-related tasks with LLMs in the literature [14, 65, 88, 92]: *natural language description* [88, 92] and *code language description* [14, 65, 68]. The former describes the task and code features in natural language, while the latter makes the prompt a code fragment by introducing the task and code features in corresponding comments. In our study, we investigated the influence of the two styles on the effectiveness of each studied open-source LLM in unit test generation.

Code Features. To adequately investigate the influence of different code features, we surveyed the existing studies on LLM-based unit test generation [8, 14, 68, 88, 92], and collected all their used code features. In total, we collected six code features and categorized them into three types: (1) *Focal Method* is the target method to be tested. The code features extracted from it include the method body (FM_b) and its parameters (FM_p). (2) *Focal Class* is the class containing the focal method. The code features extracted from it include its constructor (FC_c), the fields defined within the class

(FC_f), and other methods defined within the class (FC_m). (3) *Related Classes* are the classes including the constructors of the parameters in the focal method (except the focal class). The code feature extracted from it is these constructors (RC_c). Among them, FM_b is the basic content in the prompt for LLM-based unit test generation. Thus we performed an ablation experiment to investigate the contribution of each of the remaining five code features.

2.4 In-Context Learning Method

Chain-of-Thoughts (CoT) [92] and *Retrieval Augmented Generation* (RAG) [41] are two widely-used ICL methods in various code-related tasks. In this study, we adapted them to unit test generation task, and investigated the impact of these methods on the effectiveness of unit test generation.

CoT was originally proposed to improve the reasoning ability of LLMs in addressing mathematical and common-sense reasoning tasks [81]. Subsequently, it has been applied to some code-related tasks and the effectiveness has been also demonstrated through extensive experiments [42, 92]. Particularly, the existing work has demonstrated the effectiveness of applying CoT to ChatGPT for unit test generation [92], which instructs ChatGPT to (1) understand the focal method and then (2) generate unit tests according to its understanding. However, it is unclear whether CoT is still effective in improving open-source LLMs in unit test generation. To fill the gap, this work performs the first study that extensively investigates the effectiveness of CoT on various widely used open-source LLMs.

RAG enhances LLMs by incorporating relevant information retrieved from external databases to improve prompt contents, which has been widely used in code generation [61, 77, 79]. In general, RAG first retrieves similar inputs, and then treats these inputs along with their corresponding ground-truth outputs as examples to enhance LLMs. In our study, we investigated its effectiveness in improving LLM-based unit test generation through adaptation. Specifically, it first retrieves the most similar method to the focal method within the project. Following the existing work [34, 77], we used CodeBERT to encode the semantics of methods as vectors and then measured the cosine similarity between vectors. Then, the retrieved method, along with its corresponding unit tests, was integrated into the prompt to instruct LLMs to generate unit tests.

2.5 Benchmark

We used the Defects4J 2.0 benchmark [32] to evaluate the effectiveness of LLMs in unit test generation. It consists of 835 real-world defects collected from 17 open-source Java projects and has been widely used in software testing [33], fault localization [27, 28] and program repair [29, 30, 51, 83]. In this benchmark, each defect corresponds to a buggy version and a fixed version. The fixed version indicates that the patches have been applied to certain methods and then the defect can be fixed. To balance the evaluation costs and defect detection capability investigation, we regarded each *patched* method as a focal method to be tested, instead of all methods in this benchmark. Particularly, following the testing practice and the existing work [8, 74], we just considered *public* methods as focal methods. For a bug involving multiple buggy methods, we used all the *public* methods as focal methods. In total, we collected 778 focal methods, which involve 413 defects from all 17 projects in the

benchmark. Due to the space limit, we put the numbers of focal methods and involved defects for each project at our homepage [5].

2.6 Implementation

Due to the variable inconsistent output format of LLMs, we first used the widely-used AST parser (i.e., tree-sitter[3]) to extract the generated unit tests from the outputs, rather than relying on simple text matching. Then, we integrated all test methods into a test class for subsequent evaluation. Finally, we imported the classes within the project and some common dependencies required by JDK (e.g., the `java.util` package) and JUnit (e.g., the `org.junit.Assert` package) into the test class, in order to avoid simple compilation issues caused by missing dependencies.

We developed our experimental scripts based on PyTorch 2.0.0 [1] and transformers 4.34.1 [2], and used VLLM [37] libraries to speed up our experiments. All the experiments were conducted on four servers with the same configuration, i.e., Ubuntu 18.04 LTS, Intel Xeon Gold 6240C CPU, 512GB RAM, and eight NVIDIA A100 GPUs. All of our code and data are available at our project homepage [5].

2.7 Metrics

Following the existing studies [88, 92], we leveraged the three widely-used metrics to evaluate the generated unit tests by each technique in RQ1, RQ2, and RQ3: **Compilation Success Rate (CSR)**, **Line Coverage (Cov_L)**, and **Branch Coverage (Cov_B)**. As introduced in Section 2.6, for each focal method, we integrated all the generated unit tests into a test class, and then compiled the test class. Therefore, we calculated CSR by the ratio of the number of test classes compiled successfully over the total number of test classes corresponding to all focal methods. Then, we executed each test class that was successfully compiled, and collected its achieved line coverage and branch coverage via the code coverage collection tool (i.e., Jacoco [24]). For each test class with compilation errors, we recursively remove test methods occurred in compilation error messages from the test class and re-execute the test class until it can be compiled successfully or there is no test method left in the test class. In this way, we try our best to collect as much coverage achieved as possible. Higher CSR, Cov_L , and Cov_B indicate better effectiveness in unit test generation.

Furthermore, we incorporated an additional metric in RQ4, i.e., **the Number of Detected Defects (NDD)**, to evaluate the defect detection ability of the generated unit tests. Specifically, if a test class passes on a fixed version but fails on the corresponding buggy version, it indicates that the test class detected a defect.

3 RESULTS

3.1 Influence of Prompt Design

Influence of Description Style. We first investigated the influence of description styles, i.e., natural language description (NL) and code language description (CL). Specifically, we investigated the effectiveness of each studied open-source LLM by incorporating all studied code features into the prompt in each studied description style, respectively. Table 1 shows the results in terms of CSR, Cov_L , and Cov_B . To evaluate the difference between NL and CL in statistics, we performed the Wilcoxon rank sum test [84] with a

Table 1: Effectiveness of Open-source LLMs Using Different Description Styles in Prompt.

Style	CL-7B			CL-13B			PD-34B			DC-7B			DC-33B		
	CSR	Cov _L	Cov _B	CSR	Cov _L	Cov _B	CSR	Cov _L	Cov _B	CSR	Cov _L	Cov _B	CSR	Cov _L	Cov _B
NL	40.31	18.76	16.67	53.86	18.55	17.11	51.93	28.59	23.51	47.36	23.79	18.66	49.07	26.98	23.91
CL	34.37	16.21	14.34	38.95	14.16	12.18	56.06	30.31	24.14	50.27	23.94	17.20	50.54	32.36	26.76

significance level of 0.05 between them and calculated the Rank-biserial correlation [58] scores to show the effect size. If a p -value is smaller than 0.05, it indicates that there is a statistically significant difference between NL and CL for the used LLM in terms of the metric. An *effect-size value* above 0.3 generally indicates a meaningful difference between compared groups [13]. In this table, we used bold to highlight the better one with statistical and meaningful significance according to the p -value (<0.05) and the effect size (>0.3) between NL and CL.

From Table 1, CL-7B and CL-13B perform better when using the NL style than using the CL style in terms of all three used metrics. In particular, the superiority of NL over CL is statistically significant in all six cases for CL-7B and CL-13B. For example, the unit tests generated by CL-7B in the NL style achieve 2.55% and 2.33% higher line coverage and branch coverage than those in the CL style. For DC-7B, DC-33B, and PD-34B, there is no statistically significant difference between the two styles in most (seven out of nine) cases, even though the CL style performs slightly better than the NL style in general. The results indicate that DC-7B, DC-33B, and PD-34B are more robust with respect to the description style used in prompt than CL-7B and CL-13B.

We then investigated the reason behind the above phenomenon and found that the root cause lies in which description styles are employed in the training data for LLMs. Specifically, CL-7B and CL-13B were built based on Llama2 by using a code corpus with 500B tokens. Although they have learned how to understand code language, the base Llama2 model still has dominant impact on their ability of understanding prompts. In contrast, although PD-34B shares the Llama2 architecture, it conducted three rounds of additional training with CL data (i.e., from Llama2 to CodeLlama-34B, then Phind-v1 and v2), and thus it performs stably regardless of the used styles. Similarly, DC-7B and DC-33B were initially trained on a hybrid corpus comprising 80% data in the CL style and 20% data in the NL style, allowing them to work well with both styles.

Overall, when utilizing an LLM to address the task of unit test generation, it is preferable to design the prompt in the description style that the LLM was trained on, which is usually overlooked in practice. For sufficient comparisons, we adopted the style making the LLM perform better for each LLM in the following experiments. Specifically, we designed the prompt in the NL style for CL-7B and CL-13B and in the CL style for DC-7B, DC-33B, and PD-34B.

Finding 1: CL-7B and CL-13 perform significantly better when using the style of NL to instruct them, while PD-34B, DC-7B, and DC-33B perform stably regardless of the used styles. The underlying reason lies in whether the description styles used for instructing LLMs are well aligned with those used in the training data for LLMs.

Table 2: Effectiveness of Unit Tests Generated By Open-Source LLMs with Different Prompt Variants in Terms of Compilation Success Rate (CSR).

Variants	CL-7B	CL-13B	PD-34B	DC-7B	DC-33B
All	40.31	53.86	55.49	49.22	45.65
w/o FC_c	+0.99	-1.87	-0.42	-1.00	-2.43
w/o FC_f	+0.28	-3.43	+0.29	-0.86	-0.29
w/o FC_m	-2.14	-14.92	-6.42	-11.42	-4.28
w/o FM_p	+3.99	-1.01	+1.57	+1.84	-0.71
w/o RC_c	+3.99	-1.01	+1.57	+1.84	-0.71

Influence of Code Features. We performed an ablation experiment to investigate the influence of code features (i.e., FM_p , FC_c , FC_f , FC_m , and RC_c presented in Section 2.3). Specifically, we created five variants for prompt design, each of which removes a code feature from the prompt incorporating all studied code features, respectively. Then, we investigated the effectiveness of each studied open-source LLM with each of the prompt variants in its description style determined before. Table 2 shows the results in terms of CSR and Table 3 shows the results in terms of Cov_L and Cov_B. In both tables, Rows “All” present the results of each studied LLM with the prompt incorporating all studied code features, and the remaining rows present the increased/decreased effectiveness over the corresponding effectiveness shown in Rows “All” for each studied LLM. We also used bold to highlight the increment/decrement with statistical significance (p -values smaller than 0.05 and *effect size* greater than 0.3).

From Table 2, all the cases with statistical significance are at Row “w/o FC_m ” and all of them are decrement (ranging from 2.14% to 14.92%) in terms of CSR. The results indicate the significant contribution of FC_m (other methods defined within the focal class) to generate syntactic valid unit tests with LLMs. The reason may be that building a unit test scenario often requires several steps and some necessary information exists in FC_m . Hence, removing FC_m may result in significant hallucination issues for LLMs, harming the validity of generated unit tests. The remaining code features have no statistically significant influence on LLM-based unit test generation in terms of CSR, which is also as expected. Specifically, (1) regarding FC_f , fields are seldom directly accessed from unit tests; (2) regarding FC_c and RC_c , LLMs typically use no-parameter constructors for instantiation or employ mocked instances instead (e.g., about 58.49% of generated test classes involve such instantiation methods on average); (3) the parameters of the focal method also occur in the method body (i.e., the basic content FM_b that is always retained in the prompt).

Table 3: Effectiveness of Unit Tests Generated By Different Prompt Variants in Terms of Test Coverage.

Variants	CL-7B		CL-13B		PD-34B		DC-7B		DC-33B	
	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>
All	18.76	16.67	18.55	17.11	33.41	27.12	29.02	23.77	32.72	29.26
w/o FC_c	+8.09	+7.66	-2.44	-0.12	+0.62	+1.62	-2.72	-2.01	-3.16	-2.31
w/o FC_f	+7.46	+6.52	+1.83	+0.92	+5.17	+4.61	-0.37	-0.75	+1.11	-0.70
w/o FC_m	+9.26	+8.67	+3.07	+3.62	+4.04	+5.23	-0.65	+2.27	+1.79	+0.05
w/o FM_p	+8.93	+8.09	-0.32	+0.37	+2.46	+2.57	-1.31	-1.18	-1.63	-1.91
w/o RC_c	+8.96	+8.11	-0.32	+0.37	+2.45	+2.56	-1.31	-1.18	-1.90	-2.10

From Table 3, we found a surprising phenomenon, i.e., almost all cases at Row “w/o FC_m ” are increment, even with statistical significance. This indicates that removing FC_m in prompt design is helpful to improve code coverage even though it is harmful to the syntactic validity of generated unit tests. We carefully analyzed the underlying reason and found that the number of generated unit tests may be the key factor. Specifically, these open-source LLMs have fixed-size space to include prompt contents and LLMs’ response. If the prompt is longer, it indicates that the contents of the response from the LLM are fewer, potentially leading to a smaller number of generated unit tests. After removing FC_m in prompt design, the window space left for the LLM’s response becomes larger, and indeed, the average number of generated unit tests across all the studied open-source LLMs are significantly increased from 3,654 to 5,434, which is indeed helpful to improve code coverage. Overall, this benefit even outperforms the negative influence on the syntactic validity of generated unit tests.

Finding 2: FC_m contributes the most to the syntactic validity of generated unit tests among all studied code features, but its extensive length limits the number of generated unit tests with LLMs. The outperforming influence of the latter over the former finally makes FC_m limit test coverage.

Except FC_m , removing each of the remaining four code features is significantly helpful to improve code coverage for the three LLMs based on CodeLlama in general, but has no statistical significant influence on the two LLMs based on DeepSeek-Coder in terms of code coverage. The underlying reason may lie in the ability of the foundation models in comprehending code features. Specifically, the CodeLlama models are trained from Llama2, which are pre-trained on natural language data, while the DeepSeek-Coder models are built from scratch based on a corpus primarily composed of code language. Therefore, DC-7B and DC-33B are able to comprehend provided code features more sufficiently, facilitating generating effective unit tests. Regarding CL-7B, CL-13B, and PD-34B, because of the relatively weak ability in comprehending code features, the negative influence of missing some information due to removing code features is smaller than the brought benefit (i.e., leaving larger space for LLMs’ response), eventually bringing the improvement of test coverage. Additionally, the possible reason why removing FC_m has positive influence on both CodeLlama and DeepSeek-Coder models is that FC_m is significantly longer than the other code features, and thus the benefit in leaving space for LLMs’ response

due to its removal is more significant to test coverage improvement by generating more unit tests. Overall, there seems to be a trade-off between including more code features and leaving space for generating more unit tests in prompt design.

Particularly, when incorporating all studied code features, the relatively small CodeLlama models (i.e., CL-7B and CL-13B) perform worse than the relatively small DeepSeek-Coder model (i.e., DC-7B) in terms of code coverage, and the relatively large CodeLlama model (i.e., PD-34B) performs closely to the relatively large DeepSeek-Coder model (i.e., DC-33B). However, in our ablation experiment, the best effectiveness of CL-7B and CL-13B in terms of code coverage (i.e., that achieved by the prompt variant removing FC_m) is even better than that of DC-7B. Similarly, the achieved code coverage with PD-34B using the prompt variant removing FC_m outperforms the best effectiveness of DC-33B. This indicates the importance of selecting code features for a given LLM due to the diverse characteristics of LLMs. For sufficient comparisons, we adopted the prompt variant making the LLM perform the best for each LLM in the following experiments. Specifically, we incorporated all studied code features in the prompt for DC-7B and DC-33B but removed FC_m for the remaining three LLMs. Note that the prompt variant performing the best in our ablation experiment may be not the global optimum, since we did not evaluate all combinations of these code features due to significant costs. We will discuss this threat in detail in Section 4.

Finding 3: Code features (except FC_m) have different influence on LLMs due to different abilities of their foundation models in comprehending code features. Using proper code features may largely enhance unit test generation effectiveness of LLMs (especially CodeLlama models).

3.2 Effectiveness Comparison

Table 4 shows the effectiveness of each studied open-source LLM with the corresponding best prompt design mentioned above in terms of CSR, *Cov_L*, and *Cov_B*. To better understand the effectiveness of open-source LLMs in unit test generation, we also reported the effectiveness of traditional technique Evosuite with its default setting, as well as the closed-source GPT-4 model by incorporating all studied code features in the natural language description style (the most widely-used setting in the literature [72, 88, 92]).

Table 4: Effectiveness Comparison of Studied LLMs and Evosuite in Unit Test Generation.

Model	CSR(%)	Cov_L (%)	Cov_B (%)
CL-7B	38.17	28.02	25.34
CL-13B	38.94	21.62	20.73
PD-34B	49.07	37.45	32.35
DC-7B	49.22	29.02	23.77
DC-33B	45.65	32.72	29.26
GPT-4	52.96	40.43	31.78
Evosuite	85.71	78.91	76.59

From Table 4, the two most large-scale open-source LLMs (i.e., PD-34B and DC-33B) achieve higher code coverage than the remaining small-scale LLMs, and the commercial GPT-4 generally performs better than these studied open-source LLMs. For example, DC-33B achieves 12.75% higher line coverage and 23.10% higher branch coverage than DC-7B. This is as expected since larger-scale LLMs tend to have a stronger ability to understand instructions, and the same finding has been observed on the task of LLM-based code generation [11]. There is an unusual case, i.e., CL-13B performs worse than CL-7B in our study. This is because CL-7B suffers from a repetition issue [25], which causes that the number of generated unit tests is substantially larger than that generated by CL-13B. Although this repetition issue could benefit the task of unit test generation to some degree, it is actually widely-recognized as a typical aspect reflecting LLM’s poor capability [35]. There are also different findings derived from different tasks with LLMs. From Table 4, PD-34B outperforms DC-33B in terms of both line coverage and branch coverage, but the conclusion is opposite on the task of code generation based on the widely-known leaderboard [4]. This indicates the necessity of re-evaluating LLMs when a new task is specified, since the most suitable LLM may be different for different tasks due to different characteristics involved in them.

Finding 4: While these open-source LLMs have been evaluated on other tasks, not all findings can be generalized to unit test generation, especially in identifying the most effective open-source LLM for it. The state-of-the-art commercial GPT-4 is still more effective in unit test generation.

By comparing the traditional Evosuite with LLM-based unit test generation in Table 4, the former significantly outperforms all the LLM-based techniques (including the commercial GPT-4) in terms of all three used metrics. For example, the line coverage achieved by Evosuite is 78.91%, while that by GPT-4 is just 40.43%. Of course, the unit tests generated by Evosuite have always been criticized for poor readability, rather than for low test coverage [8, 92]. The poor readability of traditional techniques is also the initial motivation for LLM-based unit test generation [65, 72, 92], but definitely, LLM-based unit test generation should not harm test coverage too much compared to Evosuite. One possible reason is that a substantial portion of the unit tests generated by LLMs (ranging from 34.44% to 61.78%) are syntactically invalid.

To potentially improve the effectiveness of LLM-based unit test generation, we further analyzed the underlying root cause of their low CSR. Specifically, there are three main types of invalid unit tests generated by these LLMs: unresolved symbol errors, parameter mismatch errors, and abstract instantiation errors. All of them are caused by the *hallucination* issue [7, 48, 82] of LLMs. Unresolved symbol errors are the most common, which account for 30.68% of all invalid unit tests and are caused due to using semantically-coherent but undefined identifiers in unit tests. Parameter mismatch errors are the second most common, accounting for 17.25%, which involves incorrect parameter types or counts in invoked APIs. This is because there are many APIs sharing the same name but different parameter lists due to the overloading feature, aggregating the difficulty in generating correct API invocations for LLMs. Abstract instantiation errors are the third most common, accounting for 10.38%, which involves incorrect instantiation of abstract classes. Specifically, when a method parameter is defined in an abstract class, an instance of the corresponding subclass should be declared since abstract classes cannot be instantiated. However, LLMs often declare an instance for the parameter according to its defined type.

Finding 5: LLM’s hallucination leads to three main types of compilation errors for unit tests, i.e., unresolved symbol errors, parameter mismatch errors, and abstract instantiation errors. This makes LLM-based unit test generation underperform Evosuite in terms of test coverage, although the latter suffers from the poor readability issue.

3.3 Effectiveness of In-Context Learning

We investigated the influence of in-context learning on LLM-based unit test generation by evaluating each studied open-source LLM with the CoT and RAG methods (introduced in Section 2.4), respectively. Table 5 presents their effectiveness in terms of Cov_L and Cov_B . We also used bold to highlight the increment/decrement with statistical significance (p-values smaller than 0.05 and effect size greater than 0.3).

From this table, incorporating CoT weakens the effectiveness of all three CodeLlama models (i.e., CL-7B, CL-13B, and PD-34B) in terms of Cov_L and Cov_B , but enhances the effectiveness of both DeepSeek-Coder models (i.e., DC-7B and DC-33B). For example, incorporating CoT results in 3.04% decrement for CL-7B in terms of Cov_L , but brings 2.72% increment for DC-7B. This difference may be attributed to the ability of LLMs in code comprehension. Specifically, CoT introduces an extra task of describing the focal method for guiding subsequent unit test generation, which stresses the code comprehension ability of LLMs more explicitly. Therefore, the LLMs with a strong code comprehension ability could provide informative descriptions, thereby improving the effectiveness of subsequent unit test generation. However, the LLMs with a relatively weak code comprehension ability may not describe the focal method correctly, eventually leading to the effectiveness decrement. Through our manual analysis, the studied DeepSeek-Coder models indeed provide more accurate and informative descriptions than the studied CodeLlama models, thereby achieving better effectiveness in terms of test coverage. This indicates the superiority of the former in the code comprehension ability, which is also aligned

Table 5: Effectiveness of Open-source LLMs with Different In-Context Learning Methods in Terms of Test Coverage.

Model	CL-7B		CL-13B		PD-34B		DC-7B		DC-33B	
	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>	<i>Cov_L</i>	<i>Cov_B</i>
Base	28.02	25.34	21.62	20.73	37.45	32.35	29.02	23.77	32.72	29.26
+ CoT	-3.04	-2.79	-6.45	-4.57	-0.02	-2.06	+2.72	+4.03	+0.69	+0.48
+ RAG	-5.57	-5.85	-6.03	-5.10	-9.28	-8.78	-5.80	-4.44	-3.34	-5.72

Table 6: Defect Detection Effectiveness of Studied LLMs.

Models	CL-7B	CL-13B	PD-34B	DC-7B	DC-33B	GPT-4
NTD	41	28	63	60	62	65
NDD	12	15	30	24	33	39

with Finding 3. Hence, it is important to determine whether incorporating CoT to guide the used LLM is really helpful through a preliminary study in advance.

Finding 6: CoT is helpful to improve the effectiveness of both DeepSeek-Coder models in unit test generation while it does not benefit the three CodeLlama models due to the stronger ability of the former in code comprehension.

From Table 5, incorporating RAG makes all studied open-source LLMs perform worse in terms of both line coverage and branch coverage. For example, the decrement of *Cov_L* and *Cov_B* for CL-7B is 5.57% and 5.85%, respectively. We extensively analyzed the reason behind the negative conclusion, and found that there is a significant gap between the unit tests retrieved by the RAG method and those that LLMs excel at generating, which causes the information that LLMs learn from the retrieved examples is limited (even disruptive). For example, the average length of the retrieved unit tests is 12.10 lines of code (LOC) while that of the generated unit tests by LLMs is 5.60 LOC. Also, the average number of retrieved unit tests per focal method (i.e., 2.41) is much smaller than that of the generated unit tests by LLMs (i.e., 6.94). That is, the current retrieval method, which is restricted within the project following the practice of code generation [34, 77], is not sufficient. This indicates that setting a high-quality database specific to the task of unit test generation could be helpful to improve the effectiveness of RAG in enhancing LLM-based unit test generation.

Finding 7: The RAG method adapted from code generation is ineffective to enhance LLMs in unit test generation. This is mainly caused by the large gap between unit tests retrieved and those that LLMs excel at generating, indicating the necessity of setting a high-quality retrieval database specific to unit test generation.

3.4 Defect Detection Ability

We further investigated the effectiveness of each LLM-based unit test generation technique in terms of defect detection by executing its generated unit tests on the corresponding faulty versions. As shown before, a number of generated unit tests by each LLM cannot

be compiled successfully, causing that there are no valid unit tests for many defects. These defects cannot be detected at all. Therefore, for each LLM, we filtered out such defects in this experiment and reported the number of remaining defects that are detectable in Row “NTD” (short for the number of testable defects that have at least test classes compiled successfully on the corresponding version) in Table 6. From this row, 87.13% of defects cannot be detected due to the compilation issue on average across all studied LLMs, suggesting that improving the validity of generated unit tests by LLMs is critical to improve the defect detection ability of them.

We then reported the number of detected defects by each LLM in the row “NDD” in Table 6. We further confirmed the larger-scale LLMs (including GPT-4, PD-34B, and DC-33B) detected more defects than the remaining smaller-scale LLMs. Among the detected defects, there are two types of defect-triggering reasons: run-time exceptions and assertion errors. For example, DC-7B, PD-34B, DC-33B, and GPT-4 detected 6, 16, 17, and 19 defects triggered by assertion errors, respectively, and 18, 14, 16, and 20 defects triggered by run-time exceptions. On the other hand, all defects detected by CL-7B and CL-13B were triggered by run-time exceptions, showing their relatively weaker ability in generating proper assertions. Despite these findings, the percentage of detected defects is still small, ranging from 29.27% to 60.00%. This indicates the weak defect detection ability of LLM-based unit test generation.

Finding 8: The defect detection ability of LLM-based unit test generation is weak, mainly caused by the low validity of generated unit tests. On average, for 87.13% defects, LLMs do not generate any valid unit tests. Among the remaining defects, only 47.28% defects are exactly detected.

We then analyzed why just a few defects are detected among all detectable ones in order to further understand the challenges for LLM-based unit test generation in defect detection. Specifically, following the PIE theory in software engineering [75], we designed three reasons why these defects were not detected by generated tests. Then, four authors with over four years of Java programming experience manually analyzed and independently labeled each defect. The Cohen’s Kappa score is 0.95, indicating the labeling quality to some degree. Then, all the inconsistencies in labeling were discussed together to reach a consensus. Table 7 presents the number of undetected defects caused by each reason for each LLM.

(1) **Insufficient test coverage** refers to that all the generated unit tests by an LLM do not reach the faulty code. From Table 7, there are 3.61%, 1.81%, 6.63%, 6.02%, 5.42% of undetected defects caused by this reason for CL-7B, CL-13B, PD-34B, DC-7B, and DC-33B, respectively. (2) **Missing specific inputs** refers to that there are some unit tests executing the faulty code but the inputs specified

Table 7: Number of Defects that Valid Unit Tests Failed to Detect Categorized by Three Reasons.

Model	CL-7B	CL-13B	PD-34B	DC-7B	DC-33B	GPT-4
Insufficient Test Coverage	6	3	11	10	9	0
Missing Specific Inputs	22	10	21	26	20	24
Improper Assertions	1	0	1	0	0	2

in the unit tests cannot trigger the defect. That is, the triggering of the defect requires some specific inputs (such as setting the real part of a complex number to NaN for triggering the defect in Math-53). From Table 7, there are 13.25%, 6.02%, 12.65%, 15.66%, 12.05%, 14.46% of undetected defects caused by this reason for CL-7B, CL-13B, PD-34B, DC-7B, DC-33B, and GPT-4, respectively. (3) **Improper assertions** refer to that the defect-triggering inputs have been specified in some unit tests, but no proper assertions are produced to capture this defect. For example, the defect in Compress-34 lies in missing to flush the input stream after closing a file. However, the unit tests generated by PD-34B do not contain the assertions checking whether the corresponding attribute is null, but just check whether the file is closed. From Table 7, there are 0.60%, 0.60%, 1.20% of undetected defects caused by this reason for CL-7B, PD-34B, and GPT-4, respectively.

Finding 9: Regarding the undetected defects with valid unit tests generated by LLMs, the main reason lies in missing to produce the specific inputs required to trigger defects, resulting in 74.99% of undetected defects on average.

4 THREATS TO VALIDITY

The threats to *internal* validity mainly lie in our experimental scripts and the usages of LLMs. To reduce the former threat, two authors have carefully checked our implementations through code review and testing. We also released our implementations at our project homepage for replication. To reduce the latter threat, we used publicly accessible model weights from Hugging Face adhered strictly to the well-documented usage guidelines for open-source models, and leveraged the API services provided by OpenAI for GPT-4.

The threats to *external* validity mainly lie in our used benchmark and LLMs. In our study, we adopted the Defects4J benchmark for evaluation, which has been widely used in the existing studies on software testing and debugging [27, 28, 47, 86]. Specifically, we collected 778 focal methods involving 413 defects from 17 projects, exhibiting the diversity of our evaluation data to some degree. Also, we studied five open-source LLMs and the state-of-the-art commercial LLM (i.e., GPT-4). As presented in Section 2.2, we selected these LLMs by considering diverse architectures and model sizes, as well as their effectiveness on code-related tasks highlighted in the leaderboard hosted on Hugging Face. The above diversity is helpful to reduce this kind of threats to some degree. Actually, the current experiments on the used benchmark and LLMs have spent 3,000 A100 GPU hours, and our experimental design balances conclusion generalizability and evaluation costs well.

The threats to *construct* validity mainly lie in our studied code features, ICL methods, randomness, and potential data leakage. Regarding the former, we collected all the code features that have been

used in the existing studies on LLM-based testing [8, 74, 88, 92], demonstrating the comprehensiveness of our investigation. However, due to the evaluation costs, it is unaffordable to investigate the effectiveness of all combinations of these code features. Instead, we performed an ablation experiment by respectively removing one code feature from the whole set to investigate the contribution. Therefore, we may not find the *globally* optimal setting for each studied LLM for the experiments in Sections 3.2, 3.3, and 3.4, but selected the *locally* optimal setting from our ablation experiment.

Regarding the ICL methods, we investigated two most widely-studied methods, i.e., CoT and RAG, in the literature [41, 81]. According to the characteristics of unit test generation, we adapted them to fit our task as introduced in Section 2.4. The current adaptations are reasonable but may be not the best strategies in unit test generation. In the future, we will explore more usages of CoT, RAG, and some other ICL methods (e.g., self-repairing [88], self-consistency [78], prompt tuning [39]) in LLM-based unit test generation for more sufficient investigation. To reduce the threat of randomness from LLMs, we set the temperature to zero, which is the most widely-used setting to increase LLMs' determinacy [19, 57]. Same as the existing studies on LLMs using the Defects4J benchmark [56, 85, 88], there is also a potential data leakage threat in our study. Following the existing practice [65, 85], we compared LLM-generated unit tests with the original unit tests equipped by this benchmark. We found that there is no exact match between them, and even the number of LLM-generated unit tests (3.70 on average) is largely different with that of original unit tests (2.41). This helps reduce the influence of this threat to some degree. In the future, we will extend our study to more recent benchmarks, such as GitHub-Java [69].

5 IMPLICATIONS

Based on our findings from the study, we summarize a series of implications, which could suggest better practice of leveraging LLMs in unit test generation.

① **Tuning prompt design (including both description styles and code features) for a given LLM is important.** As shown in Section 3.1, both description styles and code features significantly affect the effectiveness of LLM-based unit test generation. Improper prompts can even decrease effectiveness in terms of both CSR and test coverage. This suggests that, for a given LLM, it is necessary to perform preliminary experiments to determine an optimal prompt for unit test generation.

② **Removing useless information from code features helps balance prompt content and space for generating unit tests.** According to Section 3.1, there is a trade-off between incorporating more information and leaving sufficient space for generating unit tests, particularly for FC_m . Directly removing FC_m is somewhat coarse-grained and may result in the loss of necessary information

for constructing valid unit tests. Therefore, conducting static analysis to remove the methods in FC_m that do not have dependencies to the focal method is a promising method to achieve this balance.

③ Empirically selecting a proper LLM is necessary, instead of relying on the best LLM according to experience in other tasks. As shown in Section 3.2, larger-scale LLMs generally perform better than smaller-scale ones, a conclusion consistent across unit test generation and other code-related tasks. However, for LLMs with similar scales, the best one may vary depending on the specific task. This variability arises because open-source LLMs are based on different foundation models and fine-tuned on different datasets, thus exhibiting different capabilities across various tasks. Therefore, to generate high-quality tests, it is crucial to evaluate several LLMs rather than simply using the one deemed best in other tasks.

④ Refining the use of ICL methods specific to unit test generation is essential. As shown in Section 3.3, the effectiveness of CoT varies due to different LLMs' code comprehension abilities, which significantly affects their understanding of focal methods and, consequently, the effectiveness of unit test generation. Specifically, applying CoT to LLMs with weak code comprehension abilities can even decrease test coverage. Therefore, it is advisable to conduct preliminary studies to determine if employing CoT benefits the given LLM. Additionally, the existing RAG method used in code generation is ineffective for unit test generation, primarily due to the significant gap between retrieved unit tests and those generated by LLMs. This highlights the necessity of constructing a high-quality retrieval database specific to unit test generation.

⑤ Designing post-processing strategies to fix invalid unit tests can help mitigate the influence of LLMs' hallucination. Our study reveals that a significant percentage of unit tests generated by LLMs cannot be compiled successfully, thereby limiting their effectiveness in terms of both test coverage and defect detection. We analyzed the major error types for these invalid unit tests caused by LLMs' hallucination, and found that it is possible to design post-processing rules to fix these errors. For example, extracting the class inheritance relationship can help address abstract instantiation errors. Additionally, we can follow existing methods [88, 92] to instruct LLMs to fix these errors by feeding error messages back to them.

⑥ Designing mutation strategies for the inputs specified in LLM-generated unit tests is helpful in improving defect detection ability. As demonstrated in Section 3.4, many detectable defects remain undiscovered by valid unit tests generated by LLMs due to the absence of specific defect-triggering inputs. This is understandable as LLMs tend to produce more common inputs, given their inherent characteristics. Therefore, it could be promising to design mutation strategies to transform a generated unit test into a mutated one that includes the defect-triggering input. Such mutation strategies could involve (1) mutating an input to its boundary value, and (2) mutating an input to a value found in historically defect-triggering tests, which can be obtained from bug repositories.

⑦ Supervised fine-tuning (SFT) of open-source LLMs by incorporating unit test generation data may fundamentally improve effectiveness. The overall effectiveness of directly applying existing open-source LLMs to the task of unit test generation is unsatisfactory based on our results. This deficiency may stem

from these LLMs' limited knowledge acquisition regarding this task during the pre-training process. Therefore, conducting SFT specific to the task of unit test generation could be beneficial, representing a promising direction alongside the construction of a high-quality corpus tailored to this task.

6 RELATED WORK

Unit Test Generation. In the literature, a significant body of research has been dedicated to the development of automated unit test generation techniques [8, 10, 14, 43, 66, 68, 74, 88, 89, 92]. They can be divided into two main categories: traditional techniques and DL-based techniques.

Traditional techniques adopted a variety of methods to solve this problem, including symbolic execution [62, 87], search-based optimization [20], model checking [18, 21]. For instance, one of the most influential unit test generation techniques, i.e., Evosuite [20], treats this problem as a search problem and leverages the evolutionary search algorithm to find the tests achieving high code coverage. Although they can generate unit tests with reasonable coverage, they often lack readability and maintainability compared to manually written ones. As a result, it is challenging for developers to use these automatically generated tests in practice directly.

DL-based techniques exploited the power of pre-trained language models to unit test generation. They often treat this problem as a neural machine translation problem, which inputs the target methods and outputs the unit tests. For instance, Tufano et al. [74] proposed AthenaTest, which leverages the pre-trained BART Transformer [40] to generate unit tests about the given methods. However, the early techniques used smaller general-purpose pre-trained language models, thereby limiting their effectiveness.

Recently, with the huge impact of ChatGPT, researchers explored the effectiveness of LLMs in generating unit tests. Specifically, Yuan et al. [92] proposed ChatTester that iteratively generates unit tests based on conversations with ChatGPT. Lemieux et al. [38] proposed CODAMOSA that combines evolutionary search with CodeX [11] by exploiting code understanding ability of CodeX to overcome the low rate of generating coverage-improving tests with traditional methods. Li et al. [44] proposed using the subtle differences between fixed and buggy code versions to guide ChatGPT to generate failure-inducing tests. Schafer et al. [65] proposed TestPilot, which incorporates API documentation and GPT-3.5 for unit test generation. These LLM-based techniques show significant superiority over earlier DL-based ones. However, they are all based on closed-source LLMs, resulting in a lack of transparency and reproducibility. Recently, Yang et al. [89] proposed TELPA, which aims to improve the effectiveness of LLMs in unit test generation by integrating them with bidirectional method invocation analysis and counterexample sampling. However, their evaluation was limited to PD-34B, and it remains unclear how recent open-source LLMs perform in unit test generation.

Empirical Study on evaluating LLMs in Code-Related Tasks. With the rapid development in LLMs, there are multiple studies investigating the effectiveness of LLMs in code-related tasks [12, 17, 22, 26, 45, 46, 50, 52–55, 60, 70, 76, 85]. In particular, there are some empirical studies on LLM-based unit test generation [14, 65, 68, 88, 92]. For example, Yuan et al. [92] investigated the effectiveness of

ChatGPT in terms of the correctness, sufficiency, readability and usability of generated unit tests in unit test generation. Siddiq et al. [68] investigated the effectiveness of GPT-3.5 and Codex in unit test generation. Schafer et al. [65] investigated the effectiveness of GPT-3.5 in generating unit tests for Javascript language..

However, these existing studies mostly relied on fixed prompting strategies based on closed-source LLMs, neglecting the power of advanced open-source LLMs and the influence of various prompting factors (such as prompt design and ICL methods). In our study, we focus on open-source LLMs with extensive consideration of these factors, and performed the first study to demonstrate their effectiveness. Our findings provide a series of actionable suggestions to LLM-based unit test generation.

Furthermore, there are some empirical studies on evaluating LLMs in addressing other code-related tasks. For example, Du et al. [17] conducted experiments to investigate the effectiveness of LLMs in class-level code generation. Nam et al. [54] compared the code understanding ability of LLMs when using them to help developers understand the project. Sun et al. [70] conducted experiments to investigate the effectiveness of ChatGPT in code summarization. Facundo et al. [53] summarized three types of oracles and threats that arise from automatically generating oracles using LLMs. Researchers [31, 85] conducted experiments on LLMs to investigate their effectiveness in automated program repair. Niu et al. [55] conducted a comprehensive investigation on the effectiveness of pre-trained models in six code-related tasks. Different from them, our work targets the task of unit test generation.

Test Oracle Generation Due to the importance of test oracles, numerous research efforts have been dedicated to automatically generating test oracles for test cases [16, 71, 80, 90]. For example, Watson et al. [80] proposed ATLAS, which adopted a bidirectional RNN for test assertion generation. Dinella et al. [16] introduced TOGA to infer exceptional and assertion test oracles based on the context of the focal method. Yu et al. [90] first proposed an IR-based method for test assertion generation, and Sun et al. [71] further developed EDITAS to improve IR-based test assertion generation by considering the differences between the retrieved and target “focal-test” method pairs. In our study, we use LLMs to generate complete unit tests, including both test prefixes and test oracles. When measuring the effectiveness of generated unit tests (e.g., validity and defect-detection effectiveness), we also considered both test prefixes and oracles.

7 CONCLUSION

This work conducted an comprehensive evaluation of five open-source LLMs for unit test generation. We investigated several aspects: the influence of different prompt designs, the comparison effectiveness among open-source LLMs, the commercial closed-source LLM (i.e., GPT-4) and the traditional method (i.e., Evosuite), the influence of in-context learning on LLM-based unit tests generation, and the defect detection ability of LLM-generated unit tests. Our findings first point out that prompt design is crucial to the effectiveness of LLMs in unit test generation, and suggest that it is better to align the description style with the training data and choose code features considering the LLMs’ code comprehension ability and the prompt length. Second, our results indicate that the

conclusions drawn from other tasks do not necessarily generalize to unit test generation, especially in identifying the best open-source LLM specific to the task. However, all studied LLMs (including the best GPT-4) still underperform Evosuite, primarily due to the significant percentage of invalid tests caused by hallucination. Our further analysis suggests that effective post-processing could help mitigate this problem. Third, we found that directly adapting ICL techniques from other tasks does not improve the effectiveness of unit tests generation, requiring special design for the use of ICL methods according to the characteristics of the task. Finally, we found that the defect detection ability of LLM-generated unit tests is limited, primarily due to their low syntactic validity and the lack of specific input for triggering the defects. Our analysis indicates that designing effective mutation strategies for the inputs could further improve the defect detection effectiveness.

ACKNOWLEDGMENTS

We gratefully acknowledge the financial support provided by the National Natural Science Foundation of China (Grant Nos. 62322208, 62202040, 62232001, 12411530122) and CCF-Huawei Populus Grove Fund which made this research possible. We would like to express our sincere appreciation to the anonymous reviewers for their constructive suggestions, which have greatly improved the quality of this paper.

REFERENCES

- [1] Accessed: 2023. PyTorch. <https://pytorch.org/>.
- [2] Accessed: 2023. Transformers. <https://github.com/huggingface/transformers/>.
- [3] Accessed: 2023. Tree-sitter. <https://tree-sitter.github.io/tree-sitter/>.
- [4] Accessed: 2024. Big Code Models Leaderboard. <https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard>.
- [5] Accessed: 2024. HomePage. <https://github.com/LeonYang95/LLM4UT>.
- [6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [7] Lakshya A. Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. 2023. Monitor-Guided Decoding of Code LMs with Static Analysis of Repository Context. In *NeurIPS*.
- [8] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2023. A3Test: Assertion-Augmented Automated Test Case Generation. *CoRR* abs/2302.10352 (2023).
- [9] Xavier Amatriain. 2024. Prompt Design and Engineering: Introduction and Advanced Methods. *CoRR* abs/2401.14423 (2024).
- [10] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2023. Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools. *CoRR* abs/2312.10622 (2023).
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021).
- [12] Xiangping Chen, Xing Hu, Yuan Huang, He Jiang, Weixing Ji, Yanjie Jiang, Yanyan Jiang, Bo Liu, Hui Liu, Xiaochen Li, Xiaoli Lian, Guozhu Meng, Xin Peng, Hailong Sun, Lin Shi, Bo Wang, Chong Wang, Jiayi Wang, Tiantian Wang, Jifeng Xuan, Xin Xia, Yibiao Yang, Yixin Yang, Li Zhang, Yuming Zhou, and Lu Zhang. [n. d.]. Deep Learning-based Software Engineering: Progress, Challenges, and Opportunities. *SCIENCE CHINA Information Sciences* ([n. d.]), –.
- [13] Jacob Cohen. 2013. *Statistical power analysis for the behavioral sciences*. Routledge.
- [14] Arghavan Moradi Dakkel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Inf. Softw. Technol.* 171 (2024), 107468.
- [15] DeepSeek. 2023. DeepSeek Coder: Let the Code Write Itself. <https://github.com/deepseek-ai/DeepSeek-Coder>.
- [16] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
- [17] Xueying Du, Mingwei Liu, Kaixin Wang, et al. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *ICSE*. ACM, 81:1–81:13.
- [18] Eduard Paul Enoiu, Adnan Causevic, Thomas J. Ostrand, Elaine J. Weyuker, Daniel Sundmark, and Paul Pettersson. 2016. Automated test generation using model

- checking: an industrial evaluation. *Int. J. Softw. Tools Technol. Transf.* 18, 3 (2016), 335–353.
- [19] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *ICSE-FoSE*. IEEE, 31–53.
 - [20] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *SIGSOFT FSE*. ACM, 416–419.
 - [21] Angelo Gargantini and Constance L. Heitmeyer. 1999. Using Model Checking to Generate Tests from Requirements Specifications. In *ESEC / SIGSOFT FSE (Lecture Notes in Computer Science)*, Vol. 1687. Springer, 146–162.
 - [22] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
 - [23] Stefan Hackmann, Haniyeh Mahmoudian, Mark Steadman, and Michael Schmidt. 2024. Word Importance Explains How Prompts Affect Language Model Outputs. *CoRR* abs/2403.03028 (2024).
 - [24] Marc Hoffmann and Michael Ernst. 2021. JaCoCo: Java Code Coverage Library. <https://www.jacoco.org>. Accessed: 2024-05-19.
 - [25] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *ICLR*. OpenReview.net.
 - [26] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John C. Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *CoRR* abs/2308.10620 (2023).
 - [27] Jiajun Jiang, Ran Wang, Yingfei Xiong, Xiangping Chen, and Lu Zhang. 2019. Combining spectrum-based fault localization and statistical debugging: An empirical study. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 502–514.
 - [28] Jiajun Jiang, Yumeng Wang, Junjie Chen, Delin Lv, and Mengjiao Liu. 2023. Variable-Based Fault Localization via Enhanced Decision Tree. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–32.
 - [29] Jiajun Jiang, Yingfei Xiong, and Xin Xia. 2019. A manual inspection of defects4j bugs and its implications for automatic program repair. *Science china information sciences* 62 (2019), 1–16.
 - [30] Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. 2018. Shaping program repair space with existing patches and similar code. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. Association for Computing Machinery, New York, NY, USA, 298–309. <https://doi.org/10.1145/3213846.3213871>
 - [31] Shengbei Jiang, Jiabao Zhang, Wei Chen, Bo Wang, Jianyi Zhou, and Jie Zhang. 2024. Evaluating Fault Localization and Program Repair Capabilities of Existing Closed-Source General-Purpose LLMs. In *Proceedings of the 1st International Workshop on Large Language Models for Code*. 75–78.
 - [32] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4j: a database of existing faults to enable controlled testing studies for Java programs. In *ISSTA*. ACM, 437–440.
 - [33] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
 - [34] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick S. H. Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP (1)*. Association for Computational Linguistics, 6769–6781.
 - [35] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. 2019. CTRL: A Conditional Transformer Language Model for Controllable Generation. *CoRR* abs/1909.05858 (2019).
 - [36] Divya Kumar and Krishn Kumar Mishra. 2016. The impacts of test automation on software's cost, quality and time to market. *Procedia Computer Science* 79 (2016), 8–15.
 - [37] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
 - [38] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *International conference on software engineering (ICSE)*.
 - [39] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *EMNLP (1)*. Association for Computational Linguistics, 3045–3059.
 - [40] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
 - [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
 - [42] Jia Li, Yongmin Li, Ge Li, and Zhi Jin. 2023. Structured Chain-of-Thought Prompting for Code Generation. *CoRR* (2023).
 - [43] Kefan Li and Yuan Yuan. 2024. Large Language Models as Test Case Generators: Performance Evaluation and Enhancement. *CoRR* abs/2404.13340 (2024).
 - [44] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 14–26.
 - [45] Yichen Li, Yintong Huo, Zhihan Jiang, Renyi Zhong, Pinjia He, Yuxin Su, and Michael R. Lyu. 2023. Exploring the effectiveness of llms in automated logging generation: An empirical study. *arXiv preprint arXiv:2307.05950* (2023).
 - [46] Zongjie Li, Chaozheng Wang, Pingchuan Ma, Chaowei Liu, Shuai Wang, Daoyuan Wu, Cuiyun Gao, and Yang Liu. 2024. On extracting specialized code abilities from large language models: A feasibility study. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
 - [47] Jingjie Liang, Ruyi Ji, Jiajun Jiang, Shurui Zhou, Yiling Lou, Yingfei Xiong, and Gang Huang. 2021. Interactive patch filtering as debugging aid. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 239–250.
 - [48] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. *arXiv preprint arXiv:2404.00971* (2024).
 - [49] Hanmeng Liu, Zhiyang Teng, Leyang Cui, Chaoli Zhang, Qiji Zhou, and Yue Zhang. 2023. Logicot: Logical chain-of-thought instruction-tuning data collection with gpt-4. *arXiv preprint arXiv:2305.12147* (2023).
 - [50] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv preprint arXiv:2305.01210* (2023).
 - [51] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. Tbar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*. 31–42.
 - [52] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the robustness of code generation techniques: An empirical study on github copilot. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2149–2160.
 - [53] Facundo Molina and Alessandra Gorla. 2024. Test Oracle Automation in the era of LLMs. *arXiv preprint arXiv:2405.12766* (2024).
 - [54] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
 - [55] Changan Niu, Chuanyi Li, Vincent Ng, Dongxiao Chen, Jidong Ge, and Bin Luo. 2023. An Empirical Comparison of Pre-Trained Models of Source Code. In *ICSE*. IEEE, 2136–2148.
 - [56] Wendküni C. Ouédraogo, Laura Plein, Abdoul Kader Kaboré, Andrew Habib, Jacques Klein, David Lo, and Tegawendé F. Bissyandé. 2023. Enriching Automatic Test Case Generation by Extracting Relevant Test Inputs from Bug Reports. *CoRR* abs/2312.14898 (2023).
 - [57] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2023. LLM is Like a Box of Chocolates: the Non-determinism of ChatGPT in Code Generation. *CoRR* abs/2308.02828 (2023).
 - [58] Julie Pallant. 2020. *SPSS survival manual: A step by step guide to data analysis using IBM SPSS*. Routledge.
 - [59] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the diffusion of test smells in automatically generated test code: An empirical study. In *2016 IEEE/ACM 9th International Workshop on Search-Based Software Testing (SBST)*. IEEE, 5514 (2016).
 - [60] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
 - [61] Md. Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval Augmented Code Generation and Summarization. In *EMNLP (Findings)*. Association for Computational Linguistics, 2719–2734.
 - [62] Corina S. Pasareanu, Peter C. Mehltz, David H. Bushnell, Karen Gundy-Burlet, Michael R. Lowry, Suzette Person, and Mark Pape. 2008. Combining unit-level symbolic execution and system-level concrete execution for testing NASA software. In *ISSTA*. ACM, 15–26.
 - [63] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xi-aoping Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023).

- [64] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv preprint arXiv:2402.07927* (2024).
- [65] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Trans. Software Eng.* 50, 1 (2024), 85–105.
- [66] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C Gall, and Alberto Bacchelli. 2019. On the effectiveness of manual and automatic unit test generation: ten years later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 121–125.
- [67] Mohammed Latif Siddiq, Simantika Dristi, Joy Saha, and Joanna Santos. 2024. Quality Assessment of Prompts Used in Code Generation. *arXiv preprint arXiv:2404.10155* (2024).
- [68] Mohammed Latif Siddiq, Joanna C. S. Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. 2023. Exploring the Effectiveness of Large Language Models in Generating Unit Tests. *CoRR abs/2305.00418* (2023).
- [69] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A Reproducible Benchmark of Recent Java Bugs. *CoRR abs/2402.02961* (2024).
- [70] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Qunjun Zhang, Hanwei Qian, Yang Liu, and Zhenyu Chen. 2023. Automatic Code Summarization via ChatGPT: How Far Are We? *CoRR abs/2305.12865* (2023).
- [71] Weifeng Sun, Hongyan Li, Meng Yan, Yan Lei, and Hongyu Zhang. 2023. Revisiting and Improving Retrieval-Augmented Deep Assertion Generation. In *ASE*. IEEE, 1123–1135.
- [72] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2023. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *CoRR abs/2307.00588* (2023).
- [73] Hugo Touvron, Louis Martin, Kevin Stone, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR abs/2307.09288* (2023).
- [74] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers. *CoRR abs/2009.05617* (2020).
- [75] Jeffrey M. Voas. 1992. PIE: A Dynamic Failure-Based Technique. *IEEE Trans. Software Eng.* 18, 8 (1992), 717–727.
- [76] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Trans. Software Eng.* 50, 4 (2024), 911–936.
- [77] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 146–158.
- [78] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*. OpenReview.net.
- [79] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).
- [80] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *ICSE*. ACM, 1398–1409.
- [81] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
- [82] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *ESEC/SIGSOFT FSE*. ACM, 172–184.
- [83] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-aware patch generation for better automated program repair. In *Proceedings of the 40th international conference on software engineering*. 1–11.
- [84] Frank Wilcoxon. 1992. Individual comparisons by ranking methods. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 196–202.
- [85] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2022. Practical Program Repair in the Era of Large Pre-trained Language Models. *CoRR abs/2210.14179* (2022).
- [86] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*. Association for Computing Machinery.
- [87] Tao Xie, Darko Marinov, Wolfram Schulte, and David Notkin. 2005. Symstra: A Framework for Generating Object-Oriented Unit Tests Using Symbolic Execution. In *TACAS (Lecture Notes in Computer Science)*, Vol. 3440. Springer, 365–381.
- [88] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: a ChatGPT-based automated unit test generation tool. *CoRR abs/2305.04764* (2023).
- [89] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *CoRR abs/2404.04966* (2024).
- [90] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated Assertion Generation via Information Retrieval and Its Integration with Deep learning. In *ICSE*. ACM, 163–174.
- [91] Zhiqiang Yuan, Junwei Liu, Qiancheng Zi, Mingwei Liu, Xin Peng, and Yiling Lou. 2023. Evaluating instruction-tuned large language models on code comprehension and generation. *arXiv preprint arXiv:2308.01240* (2023).
- [92] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *CoRR abs/2305.04207* (2023).
- [93] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493* (2022).
- [94] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul F. Christiano, and Geoffrey Irving. 2019. Fine-Tuning Language Models from Human Preferences. *CoRR abs/1909.08593* (2019).